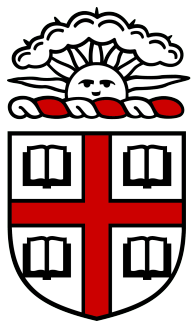
DATA1030 Final Presentation

Room Occupancy Forecasting

Yanle Lyu

Data Science Institute
Brown University

December 12, 2025



BROWN

https://github.com/LYL55555/data1030-project

Table of Contents

1. Recap
2. Cross-validation
3. Models
4. Results
5. Outlook

Research Question & Dataset Overview

Research Question

- Can we accurately predict whether a room is *occupied or not* using environmental sensor data from the current moment and several minutes earlier?

Why This Matters

- Enables energy-efficient HVAC and lighting control
- Supports smarter building automation and reduces energy waste

Task Type

- Binary classification (1 = occupied, 0 = not occupied)

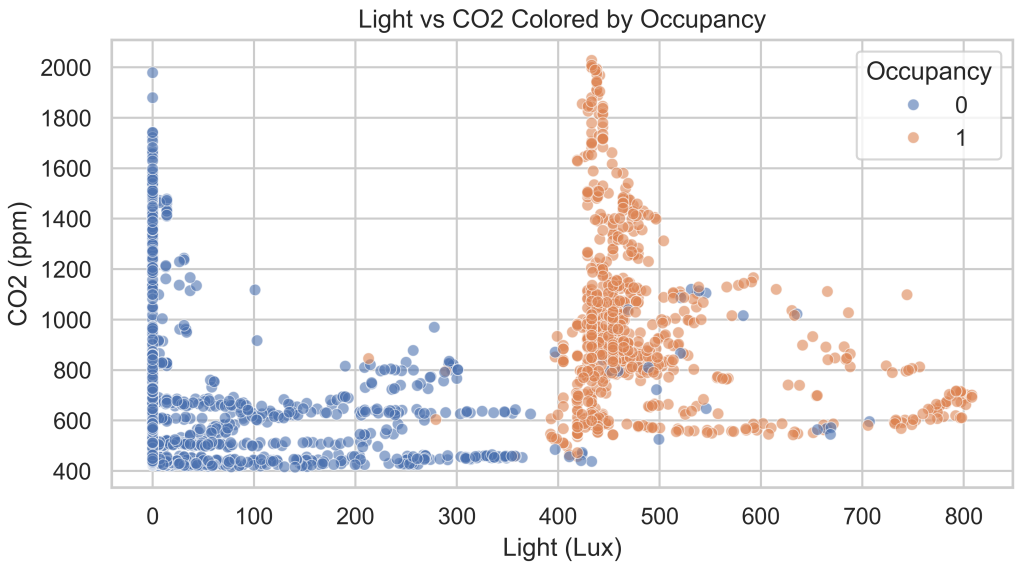
Dataset: UCI Occupancy Detection

- Recorded in a real office environment (1-minute sampling interval)
- Applied custom chronological split: 60% train / 20% validation / 20% test

Dataset Difficulty

- Non-IID time series: strong temporal autocorrelation, daily patterns → requires chronological split and time-series CV

Features & Key EDA Insights



Light vs CO_2 colored by occupancy

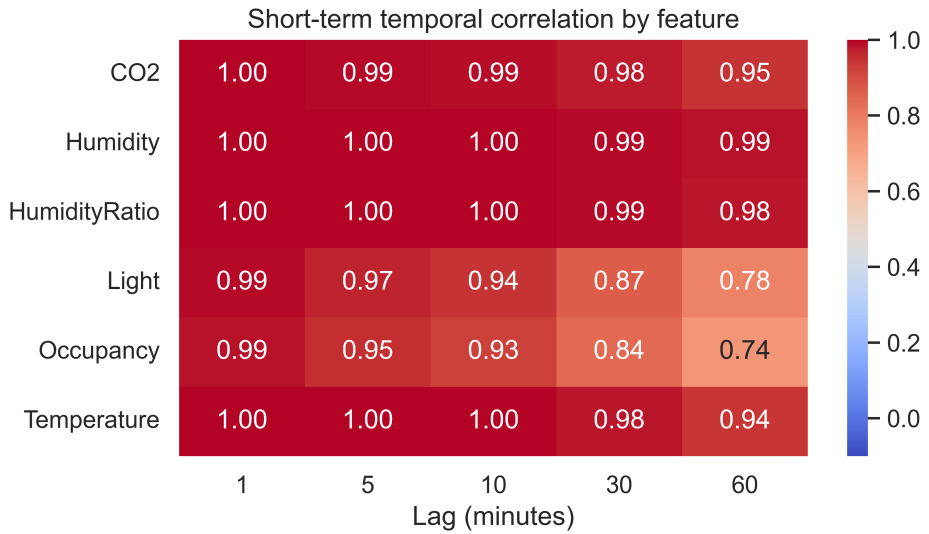
Features

- Temperature, Humidity, Light, CO_2, HumidityRatio
- Timestamp used to construct hour-of-day & day-of-week features
- Target: `Occupancy` (0 = empty, 1 = occupied)

Key EDA Findings

- In the Light–CO_2 plane, occupied and unoccupied points form almost disjoint clusters: low light and low CO_2 correspond to empty rooms, while high light and higher CO_2 correspond to occupied rooms.
- This suggests environmental sensor readings alone already contain a strong signal

Lag Features



Short-term temporal autocorrelation across features

Short-term Temporal Correlation

- Sensor readings exhibit very high autocorrelation at short lags.
- 1–10 minute lags provide almost no new information (correlation $\approx$ 0.97–1.00).
- Correlation drops substantially after 30–60 minutes.

Why 30 minutes?

- Retains strong signal (Light corr $\approx$ 0.9) but is *not redundant* with the current reading, and giving meaningful short-term trends instead of no new info.

Time-series Split & Cross-validation Pipeline

Chronological train / validation / test split

- Concatenate `datatraining.txt` and `datatest2.txt`, then sort by timestamp.
- Use a 60% / 20% / 20% time-series split because of non-iid structure:
  - Train: 10,719 rows
  - Validation: 3,573 rows
  - Test: 3,573 rows

TimeSeriesSplit for model selection

- On the training set only: `TimeSeriesSplit(n_splits=5)`.
- In each fold, earlier timestamps are used for training and later timestamps for validation.

Preprocessing & metric

- One-hot encode `day_of_week` into `dow_0`–`dow_6`.
- Fit `StandardScaler` on the training fold, then apply it to validation/test.
- Model selection metric: **F1-score**.

Dataframe Overview (Feature Matrix)

Raw sensor features

- Temperature
- Humidity
- Light
- CO_2
- HumidityRatio

Lagged features (30 minutes)

- Temperature_lag30
- Humidity_lag30
- Light_lag30
- CO2_lag30
- HumidityRatio_lag30

Temporal features

- hour_of_day
- dow_0, dow_1, dow_2, dow_3, dow_4, dow_5, dow_6

Final feature set: 18 total features

Models & Hyperparameter Grids

Baseline

- `DummyClassifier`: stratified random guesser

Logistic Regression (L1)

- Grid: `C` $\in (-3, 3, 7)$

Logistic Regression (L2)

- Grid: `C` $\in (-3, 3, 7)$

Logistic Regression (Elastic Net)

- Grid: `C` $\in (-3, 3, 7)$; `l1_ratio` $\in \{0.1, 0.5, 0.9\}$

KNN

- Grid: `n_neighbors` $\in \{1, 3, 5, 7, 9, 11, 15, 20, 25, 30, 40, 50, 100\}$; `weights` $\in \{\text{"uniform"}, \text{"distance"}\}$

SVM (RBF)

- Grid: `C` $\in (-3, 3, 7)$; `gamma` $\in (-3, 3, 7)$

Random Forest

- Grid (following lecture ranges):
- `n_estimators` $\in \{100, 200, 500\}$
- `max_depth` $\in \{1, 3, 5, 10, 30\}$
- `max_features` $\in \{0.1, 0.3, 0.5, 0.7, 0.9\}$

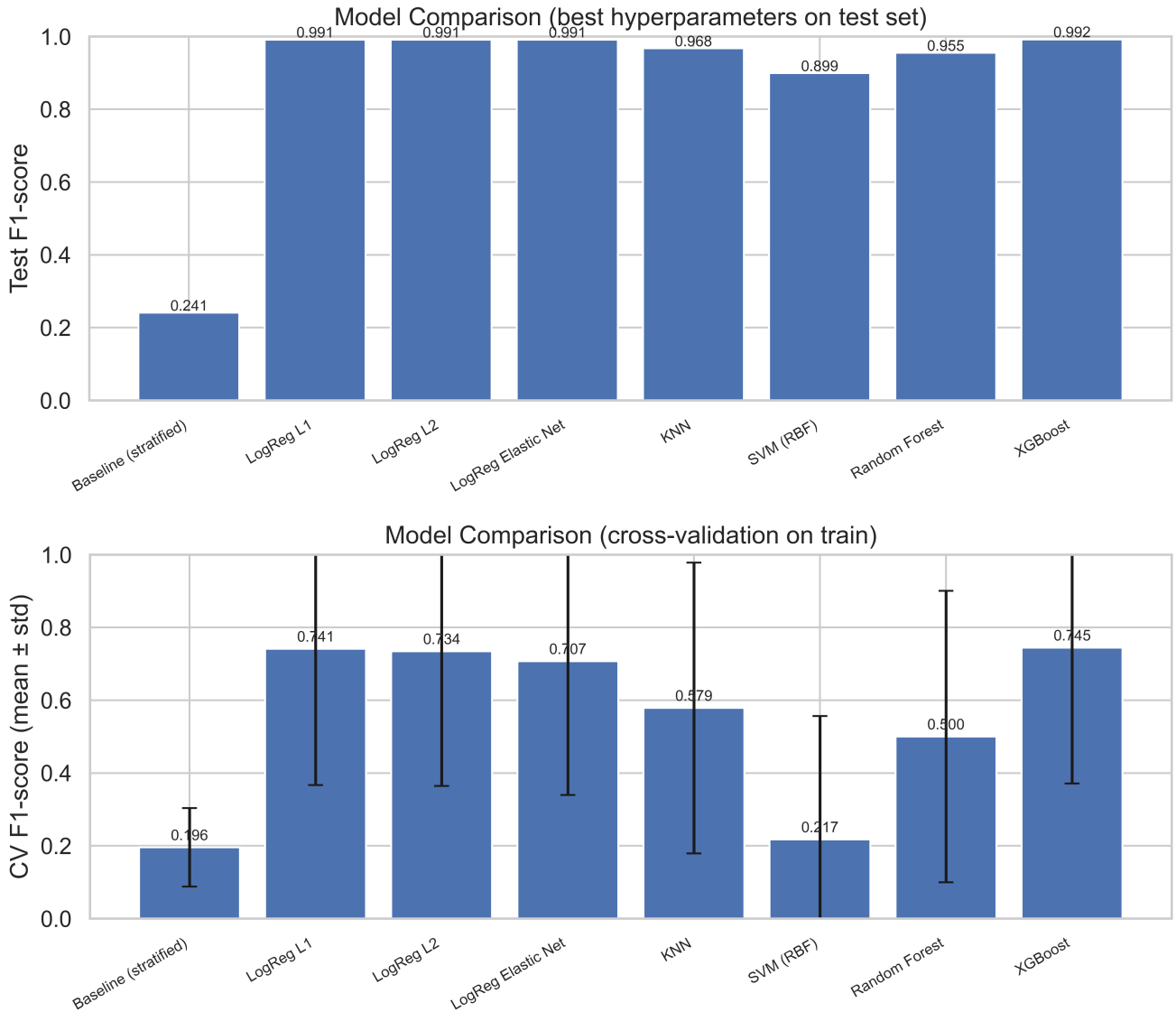
XGBoost

- Manual grid + early stopping (ranges from lecture):
- `max_depth` $\in \{1, 3, 5, 10\}$
- `reg_alpha`, `reg_lambda` $\in \{0.001, 0.01, 0.1, 1, 10, 100, 1000\}$
- `subsample`, `colsample_bytree` $\in \{0.1, 0.3, 0.5, 0.7, 0.9\}$

Pipeline

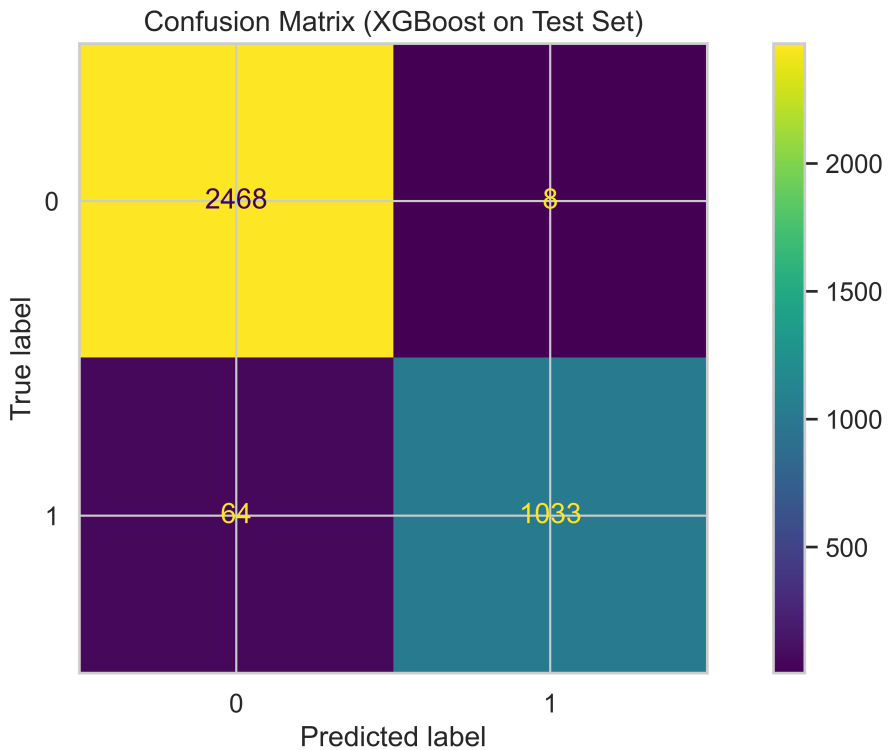
- **0–60%: Train models**
  - Use **TimeSeriesCV** for cross-validation
- **60–80%: Hyperparameter tuning**
  - Select the **best params by validation F1**
- **80–100%: Final model evaluation**
  - Compare models **only by test F1**

Model Selection: Cross-Validation and Final Test Performance

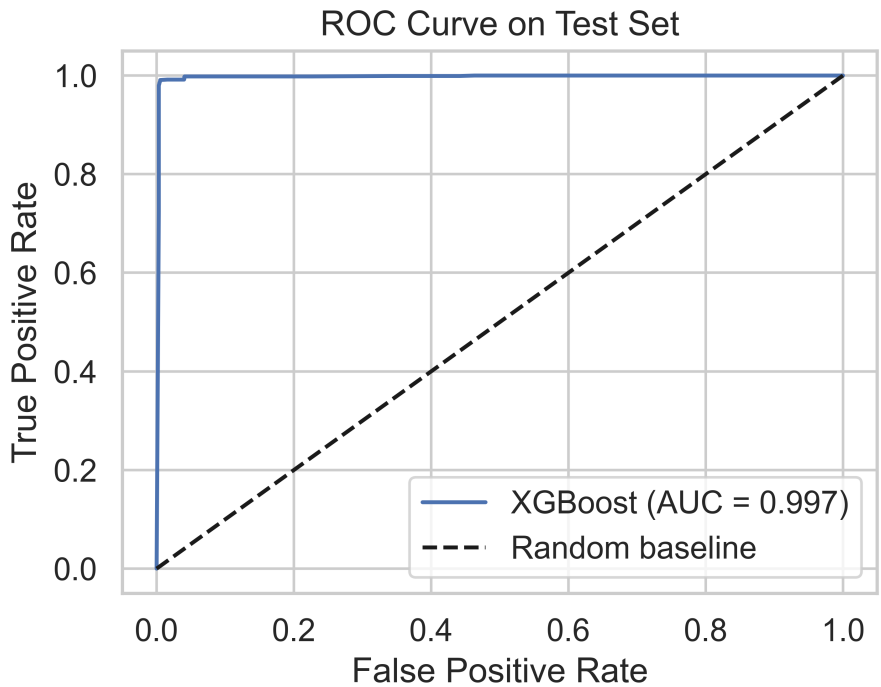


Inspecting XGBoost: Test Performance

```
Accuracy: 0.980  Precision: 0.992  Recall: 0.942  F1-score: 0.966
```

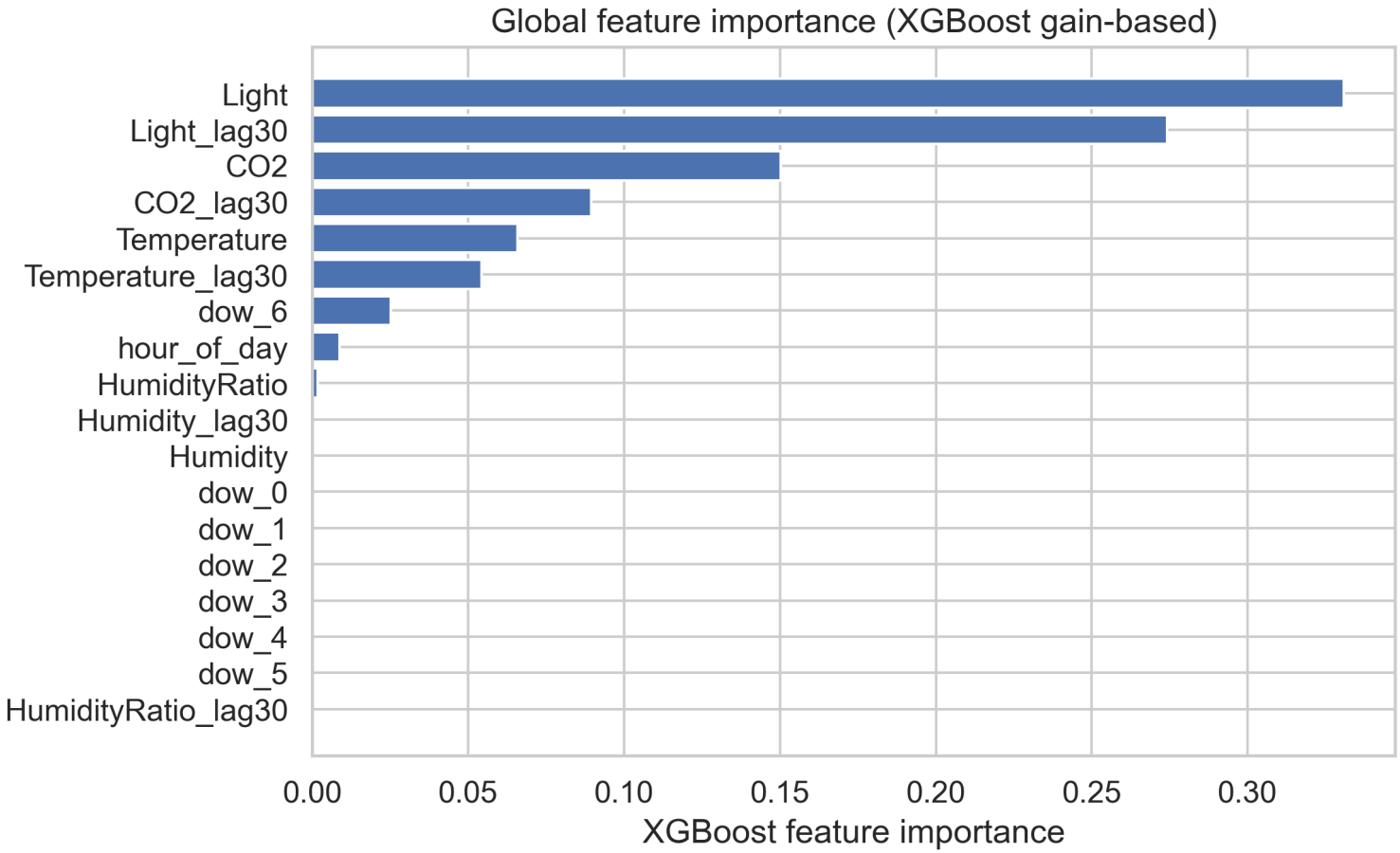


Few false positives (8) and moderate false negatives (64).



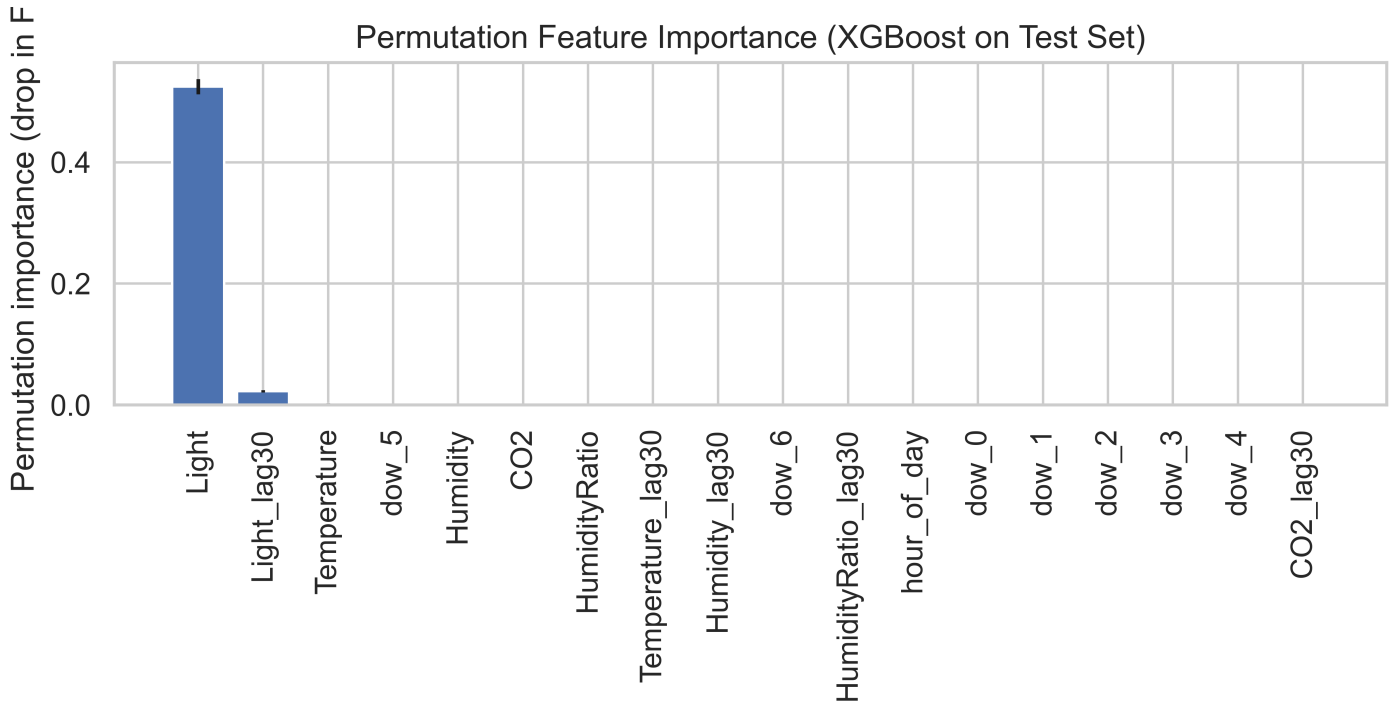
ROC AUC ≈ 0.997 shows excellent separation.

Global Feature Importance (XGBoost)



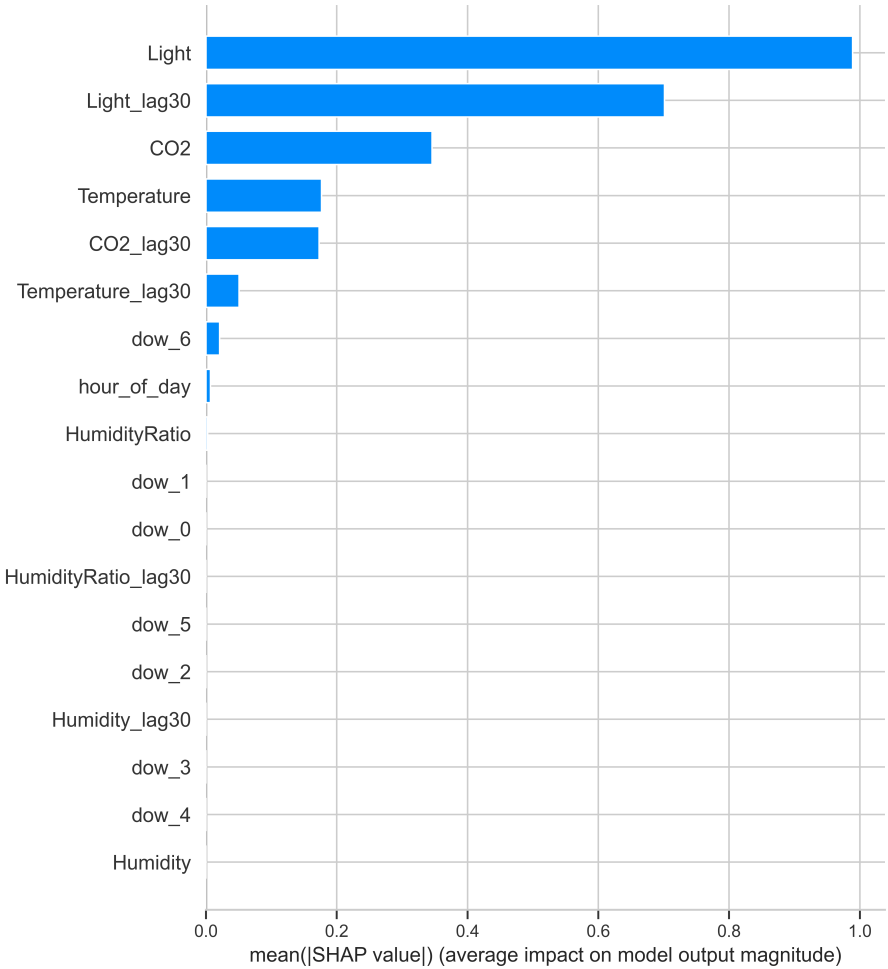
- **Light** and **Light_lag30** dominate the split gain: strong signals of occupancy.
- Time features (`dow_6`, `hour_of_day`) and humidity-related features play a smaller but non-zero role.

Global Interpretability: Permutation Importance

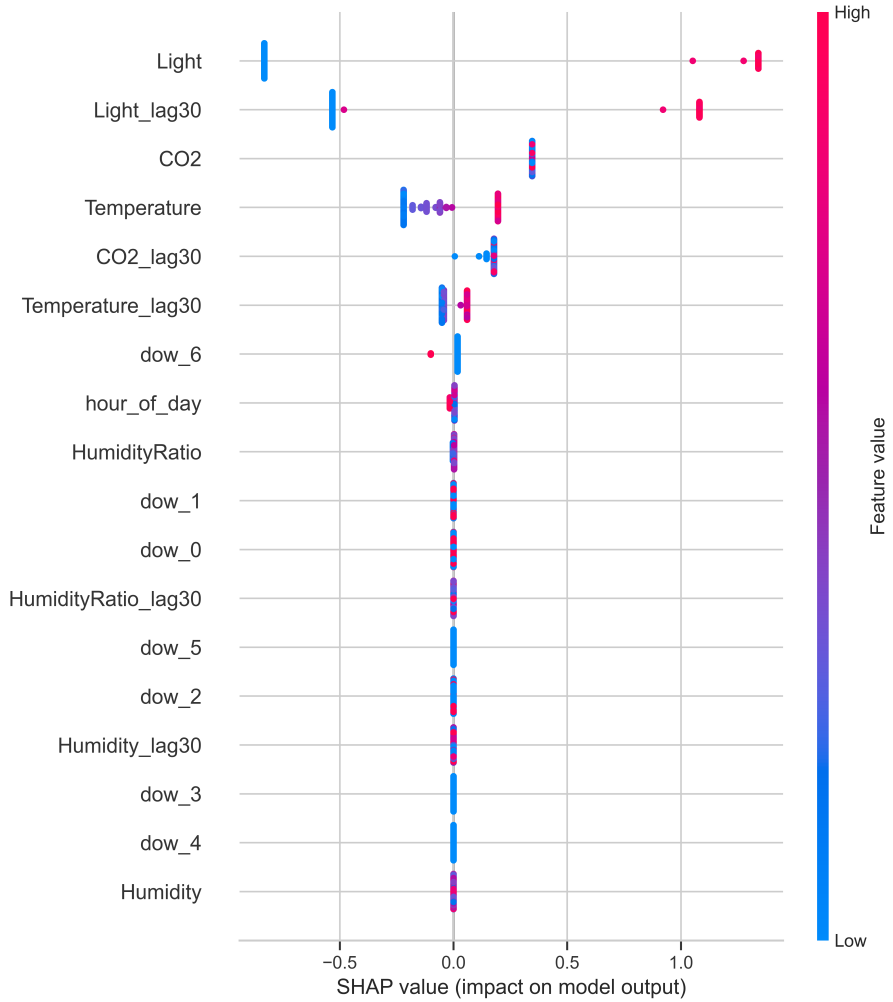


- Measures performance drop when each feature is randomly shuffled.
- **Light** and **Light_lag30** produce the largest F1 decrease, confirming they are core drivers of the model.
- Small drops for other features suggest redundancy: correlated variables can substitute for each other.

Global Interpretability via SHAP (XGBoost)



Mean |SHAP| per feature

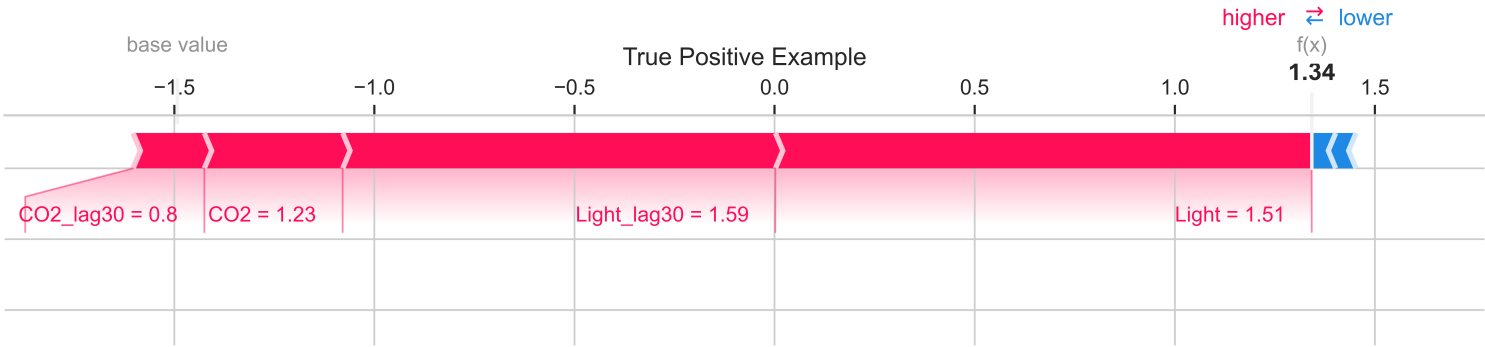


SHAP beeswarm: impact and sign

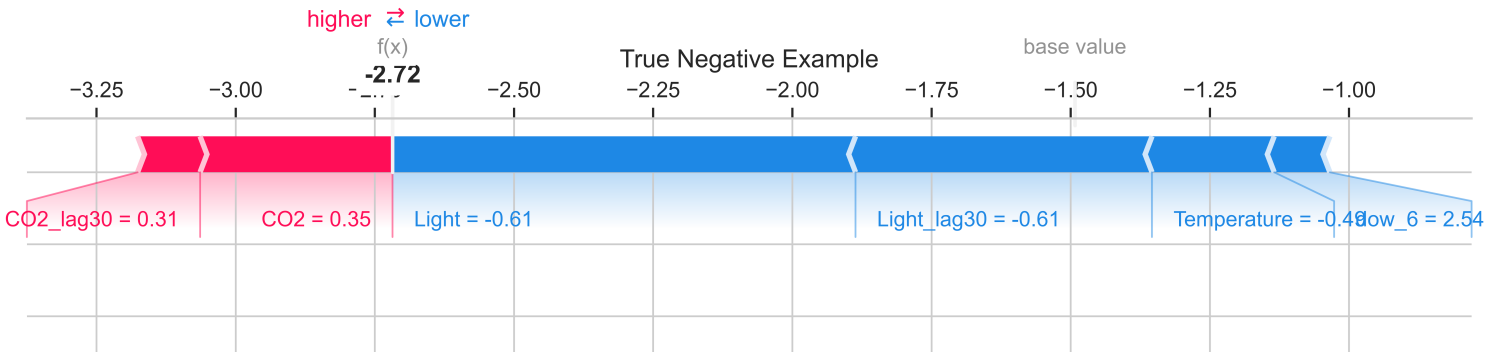
- SHAP confirms the tree-based importance: **Light**, **Light_lag30**, and **CO2** have the largest average impact.

Local Interpretability (Correct Predictions)

SHAP force plots show how each feature pushes the log-odds *higher* (red, toward occupied) or *lower* (blue, toward not occupied).



True Positive Example

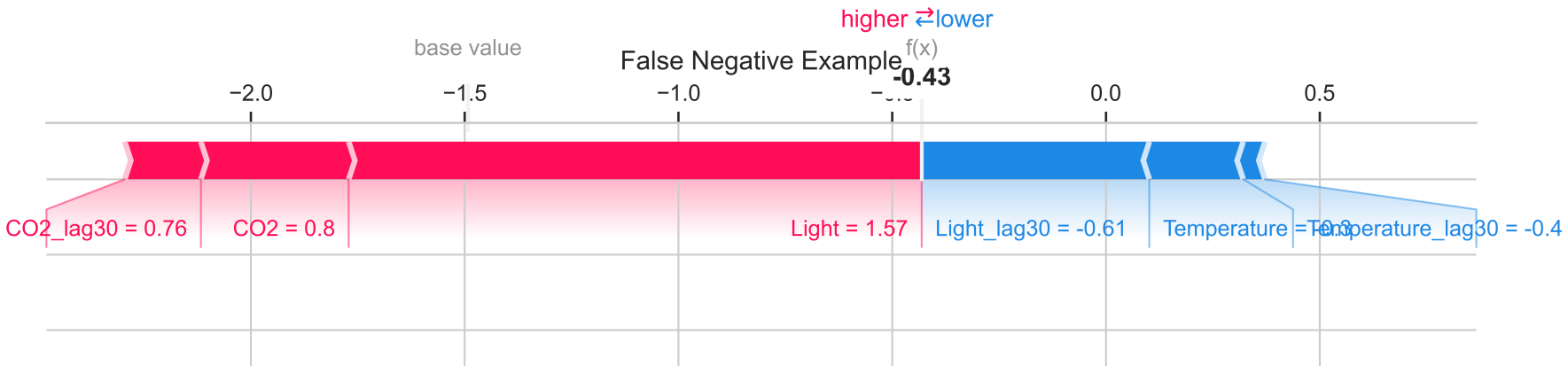


True Negative Example:

Low Light, Light_lag30, and Temperature-related features push strongly downward. Positive CO_2 contributions are too weak to offset the dominant negative evidence.

Sensor Snapshot (False Negative Example)

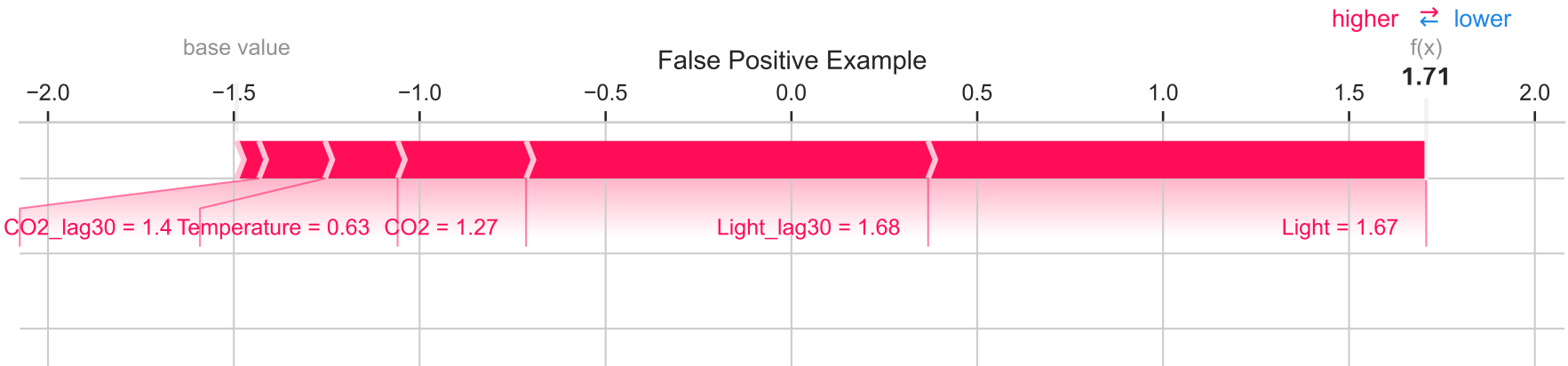
Based on the sensor readings below, would you expect the room to be *occupied* or *not occupied*?



Feature	Value
Temperature	21.70
Light	37.0
CO_2	1143.0
Light_lag30	446.0
CO_2_lag30	1209.33
Hour of day	13:00
Day of week	Tuesday (`dow_1 = True`)

Sensor Snapshot (False Positive Example)

Based on the sensor readings below, would you expect the room to be *occupied* or *not occupied*?



Feature	Value
Temperature	21.50
Light	454.0
CO_2	967.5
Light_lag30	450.25
CO_2_lag30	1006.25
Hour of day	11:00
Day of week	Monday (`dow_0 = True`)

Outlook: Improving Predictive Power and Interpretability

If I had more time, I would explore:

1. **Improving predictive power**
   - **Richer time-series features**: add longer lags and rolling statistics to better capture transitions where the current model fails (e.g., lights turned off but CO_2 still high).
   - **Ensembles over time**: combine models specialized on different time-of-day or weekday segments to handle distribution shift across the day/week.

2. **Improving interpretability**
   - **Constrain the model**: use monotonic constraints in XGBoost (e.g., CO_2 should not decrease occupancy probability) to align the model with domain intuition.
   - **Systematic error analysis**: group FNs/FPs by patterns (e.g., "dark but high CO_2" vs "bright but low CO_2") and feed this back into feature engineering.

References I

UCI Machine Learning Repository. *Occupancy Detection Dataset*.
`http://archive.ics.uci.edu/dataset/357/occupancy+detection`

Galib, M. *Occupancy Detection UCI Data (GitHub)*.
`https://github.com/galib96/occupancy-detection-uci-data`

Turksoy Omer. *HVAC Occupancy Detection with ML and DL Methods*.
Kaggle Notebook.
`https://www.kaggle.com/code/turksoyomer/hvac-occupancy-detection-with-ml-and-dl-methods`

Pandey, P. *Analyzing Machine Learning Models with Yellowbrick*.
Kaggle Notebook.
`https://www.kaggle.com/code/parulpandey/analysing-machine-learning-models-with-yellowbrick`